

SLOVENSKÁ TECHNICKÁ UNIVERZITA V BRATISLAVE
FAKULTA ELEKTROTECHNIKY A INFORMATIKY

Evidenčné číslo: FEI-104376-117218

NÁVRH IMPLEMENTÁCIE VYBRANÝCH ALGORITMOV RIADENIA

Diplomový projekt

Študijný program:	Robotika a kybernetika
Študijný odbor:	Kybernetika
Školiace pracovisko:	Ústav robotiky a kybernetiky
Školiteľ:	Ing. Marián Tárník, PhD.

Bratislava 2026

Bc. Andrii Stoliar



ZADANIE DIPLOMOVEJ PRÁCE

Študent: **Bc. Andrii Stoliar**
ID študenta: 117218
Študijný program: robotika a kybernetika
Študijný odbor: kybernetika
Vedúci práce: Ing. Marián Tárník, PhD.
Vedúci pracoviska: prof. Ing. František Duchoň, PhD.
Miesto vypracovania: Ústav robotiky a kybernetiky

Názov práce: **Návrh implementácie vybraných algoritmov riadenia**

Jazyk, v ktorom sa práca vypracuje: slovenský jazyk

Špecifikácia zadania:

Práca sa zameriava na samotnú implementáciu algoritmov riadenia pričom sa predpokladá uplatnenie poznatkov z oblasti pokročilých metód automatického riadenia v rámci príslušného študijného programu. Hlavným cieľom práce je zostavenie riadiaceho systému pre vybraný laboratórny dynamický systém vrátane vlastného softvérového návrhu pre dostupný laboratórny hardvér (mikrokontroléry, PLC a procesory pre distribuovaný riadiaci systém). Pre splnenie cieľa práce je tiež potrebné adekvátne uplatňovať postupy z oblasti identifikácie systémov, numerickej simulácie a vyhodnocovania kvality riadenia systémov.

Úlohy:

1. Oboznámte sa s dostupným laboratórnym hardvérom a vypracujte stručnú technickú dokumentáciu vybraných laboratórnych dynamických systémov.
2. Vypracujte krátky prehľad riadiacich algoritmov vybraných pre následnú implementáciu.
3. Navrhnite a realizujte predmetné riadiace systémy a analyzujte kvalitu riadenia.
4. Vyhodnoťte dosiahnuté výsledky.

Termín odovzdania diplomovej práce: 15. 05. 2026
Dátum schválenia zadania diplomovej práce: 06. 11. 2025
Zadanie diplomovej práce schválil: prof. Ing. Jarmila Pavlovičová, PhD. – garantka študijného programu

Obsah

Úvod	4
1 Regulácia tepelného systému	5
1.1 Popis tepelného systému	5
1.1.1 Popis laboratórneho fyzikálneho modelu	5
1.1.2 Statické vlastnosti tepelného systému	6
1.2 Identifikácia tepelného systému	9
1.2.1 Meranie údajov a ich spracovanie	9
1.2.2 Identifikácia systému pomocou metódy najmenších štvorcov . .	11
1.2.3 Identifikácia systému pomocou optimalizácie prenosovej funkcie	12
1.2.4 Výsledky identifikácie tepelného systému	13
1.2.5 Aproximácia identifikovaných modelov	14
1.3 PI regulátor pre tepelný systém	15
1.3.1 Štruktúra PI regulátora	15
1.3.2 Nastavenie parametrov PI regulátora	16
1.3.3 Implementácia PI regulátora v MATLABe	18
1.4 Prepínač pracovných bodov	21
1.4.1 Opis a schéma riadenia z použitím prepínača	22
Zoznam literatúry	26

Úvod

Riadenie procesov je kľúčovou súčasťou moderných priemyselných systémov, ktoré zabezpečujú efektívnu a spoľahlivú prevádzku rôznych technológií.

1 Regulácia tepelného systému

V tejto kapitole sa budeme zaoberať laboratórnym modelom, ktorý bol poskytnutý pre potreby tejto diplomovej práce – tepelným systémom TS05. Cieľom kapitoly je podrobne opísať túto experimentálnu zostavu, vykonať jej identifikáciu a navrhnúť viaceré riešenia riadenia, a to ako klasické, tak aj pokročilejšie metódy.

Súčasťou práce je tiež implementácia vybraných riadiacich algoritmov na rôznych hardvérových platformách. Táto časť má umožniť porovnanie prístupov k implementácii riadenia v simulačnom prostredí MATLAB/Simulink a na vybranom mikrokontroléri, čím sa vytvorí priestor pre analýzu rozdielov v správaní systému a v samotnej realizácii algoritmov.

1.1 Popis tepelného systému

Táto kapitola sa zaoberá popisom laboratórneho modelu TS05 a jeho základnými statickými vlastnosťami. Model predstavuje fyzický systém určený na výučbu a experimentálne overovanie princípov riadenia v oblasti automatizácie a mechatroniky.

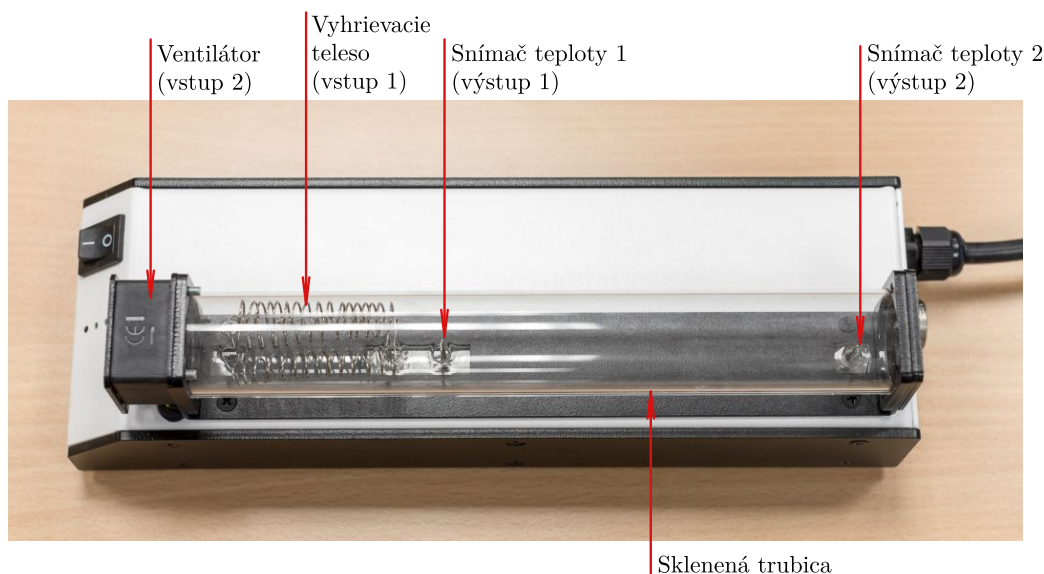
1.1.1 Popis laboratórneho fyzikálneho modelu

Laboratórny model TS05 predstavuje MIMO systém (Multiple Input – Multiple Output) so dvomi vstupmi a dvomi výstupmi, ktorého zapojenie je uvedené v tabuľke 1. Vstupné veličiny tvoria napätia riadiace ventilátor a vykurovaciu špirálu (v rozsahu 0–10 V), zatiaľ čo výstupy sú merané pomocou Senzora 1 a Senzora 2 (tiež v rozsahu 0–10 V).

Tabuľka 1: Vstupy a výstupy systému TS05

Typ signálu	Označenie	Rozsah
Vstup	Ventilátor	0–10 V
Vstup	Výkon špirály	0–10 V
Výstup	Senzor 1 (teplota 1)	0–10 V
Výstup	Senzor 2 (teplota 2)	0–10 V

Vizuálny vzhľad systému je znázornený na obrázku 1.

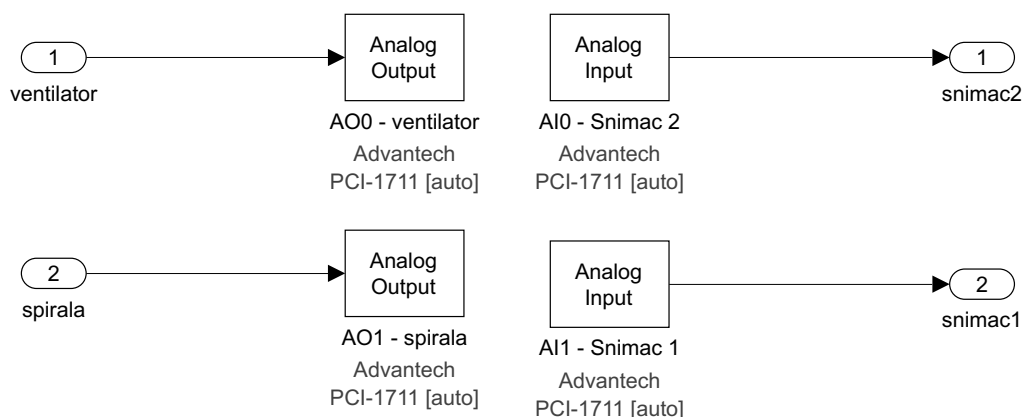


Obr. 1: Laboratórny model TS05 – reálna zostava [2]

Systém komunikuje s počítačom prostredníctvom meracej a komunikačnej karty *Humusoft*, ktorá je určená na prepojenie reálnych systémov so simulačným prostredím

MATLAB/Simulink. Táto karta umožňuje obojsmerný prenos analógových signálov – prijíma merané veličiny zo systému a zároveň vysiela riadiace signály na jeho vstupy.

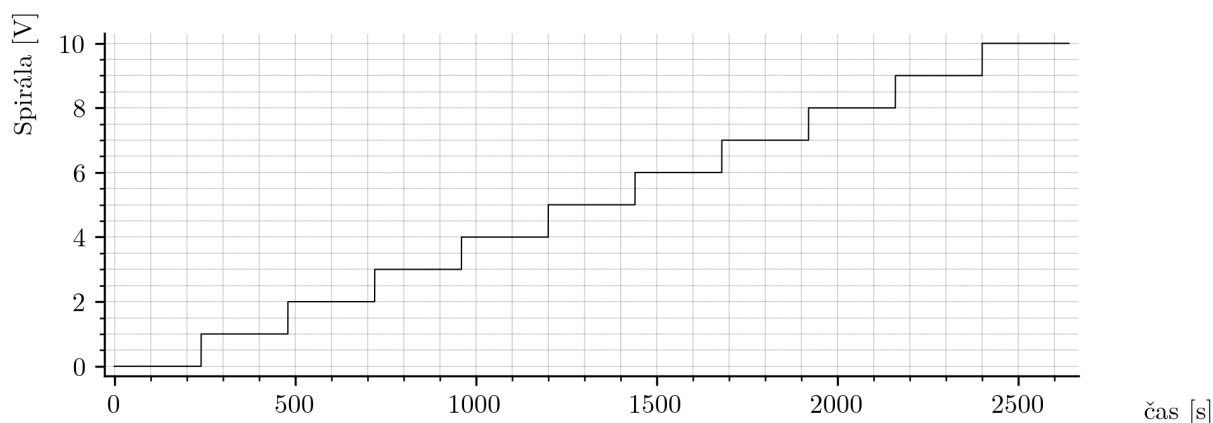
Samotná komunikácia a spracovanie signálov prebieha v prostredí Simulink, v rámci vopred pripravenej schémy (obr. 2), ktorá bola vytvorená ako súčasť projektu KUT – Krátke výučbové materiály. Táto schéma slúži aj ako podklad pre predmet *Modelovanie a riadenie systémov* [2].



Obr. 2: Simulinková schéma pre komunikáciu so systémom TS05

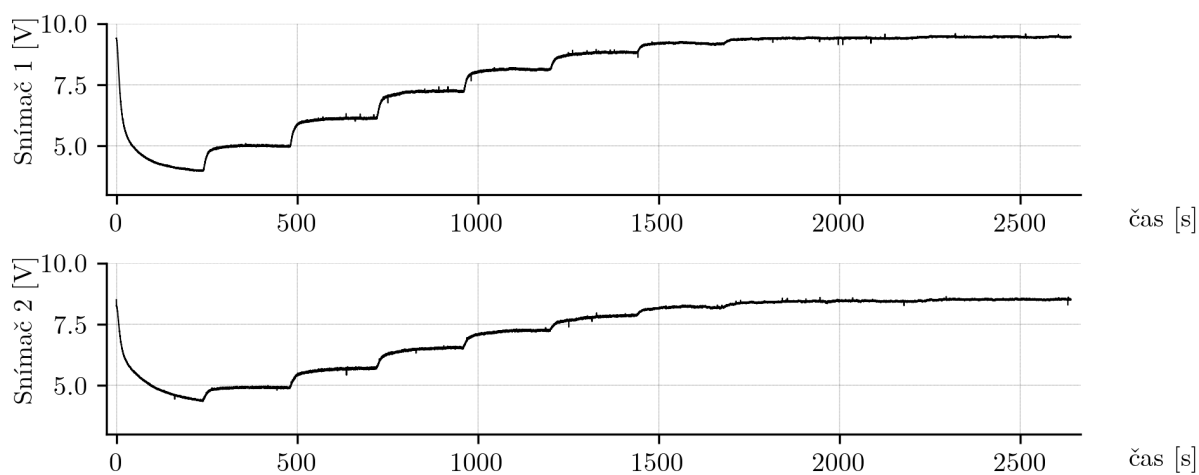
1.1.2 Statické vlastnosti tepelného systému

Na lepšie pochopenie správania systému boli experimentálne namerané jeho statické charakteristiky. Počas meraní boli skúmané tri pracovné body ventilátora – 3 V, 6 V a 9 V – pričom v každom prípade bola hodnota napätia ventilátora udržiavaná konštantná. Napätie na vykurovacej špirále sa menilo lineárne od 0 V do 10 V počas celého merania, keďže špirála má výraznejší vplyv na teplotu ako ventilátor. Priebeh signálu špirály počas experimentov je zobrazený na obrázku 3.

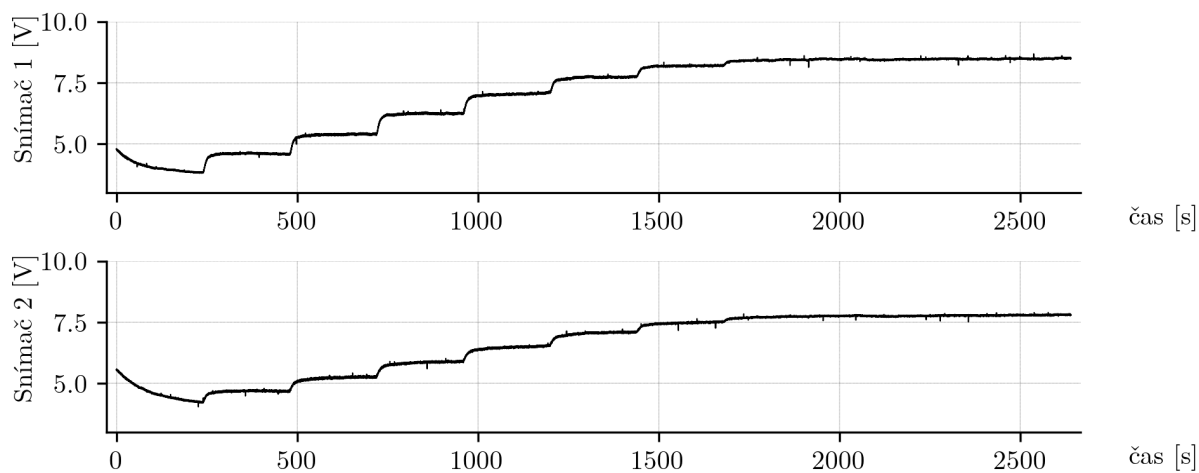


Obr. 3: Priebeh napätia na špirále počas všetkých troch experimentov

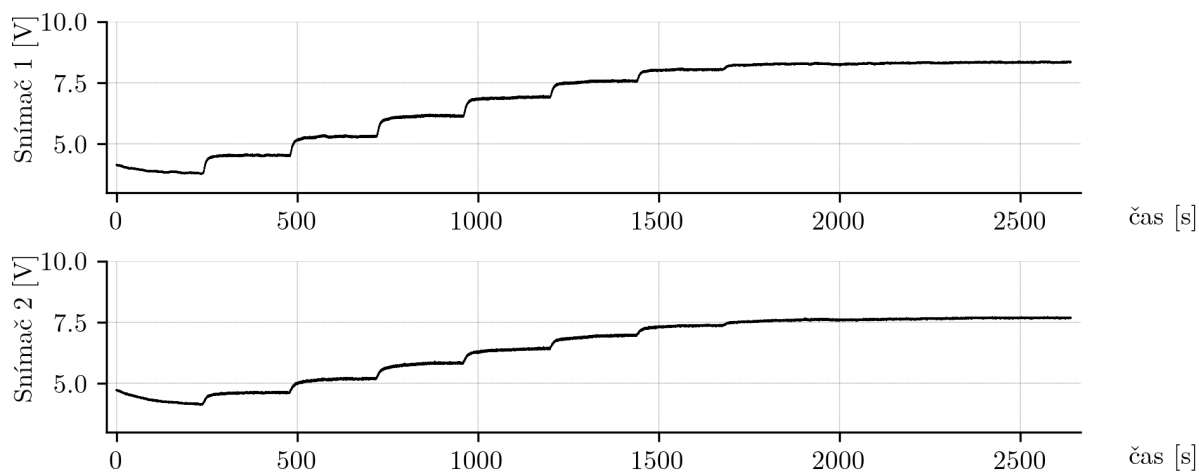
Výsledkom meraní sú šesť grafov, ktoré zobrazujú odozvu systému na zmenu vstupných veličín. Každý graf obsahuje výstupy zo Senzora 1 a Senzora 2 ako funkcie času pre jednotlivé pracovné body ventilátora.



Obr. 4: Odozva systému TS05 pre pracovný bod ventilátora 3 V

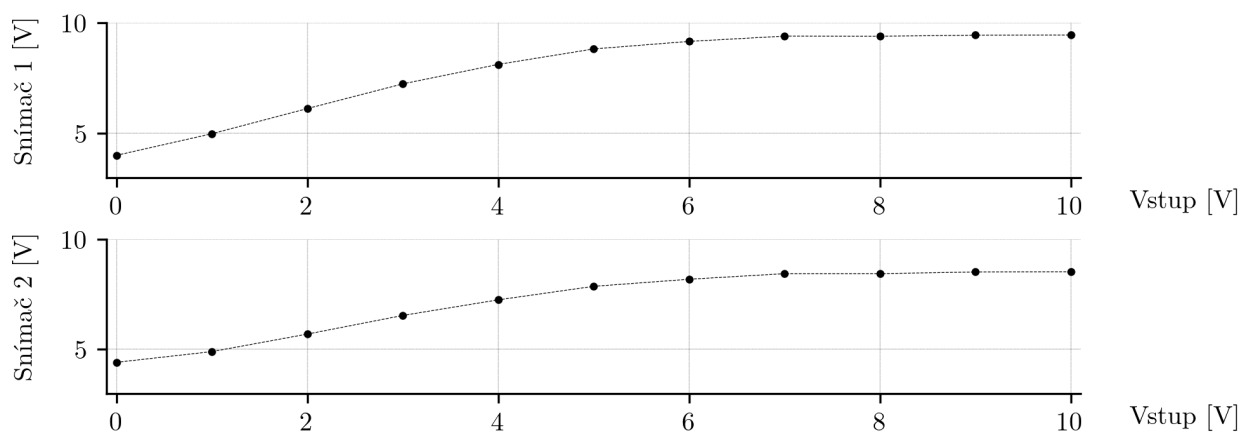


Obr. 5: Odozva systému TS05 pre pracovný bod ventilátora 6 V

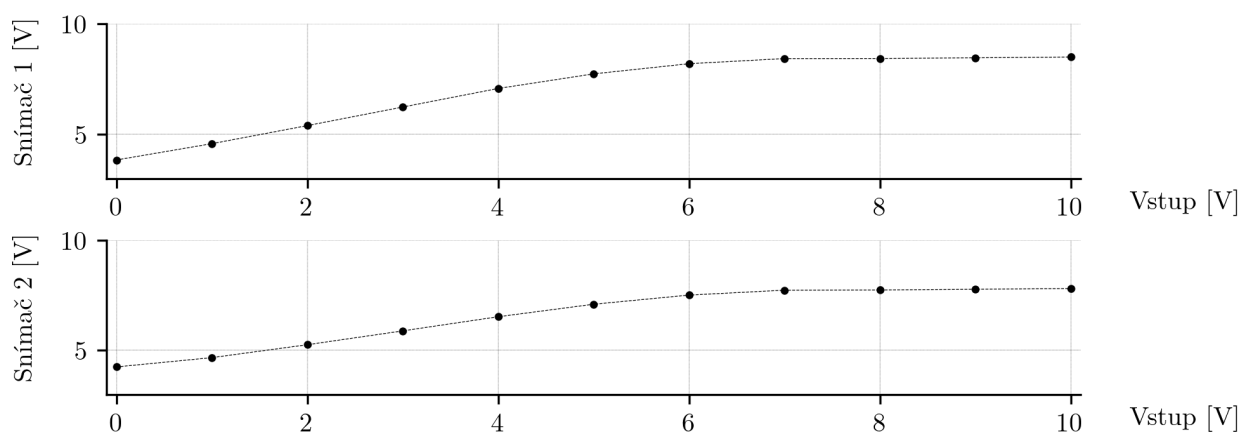


Obr. 6: Odozva systému TS05 pre pracovný bod ventilátora 9 V

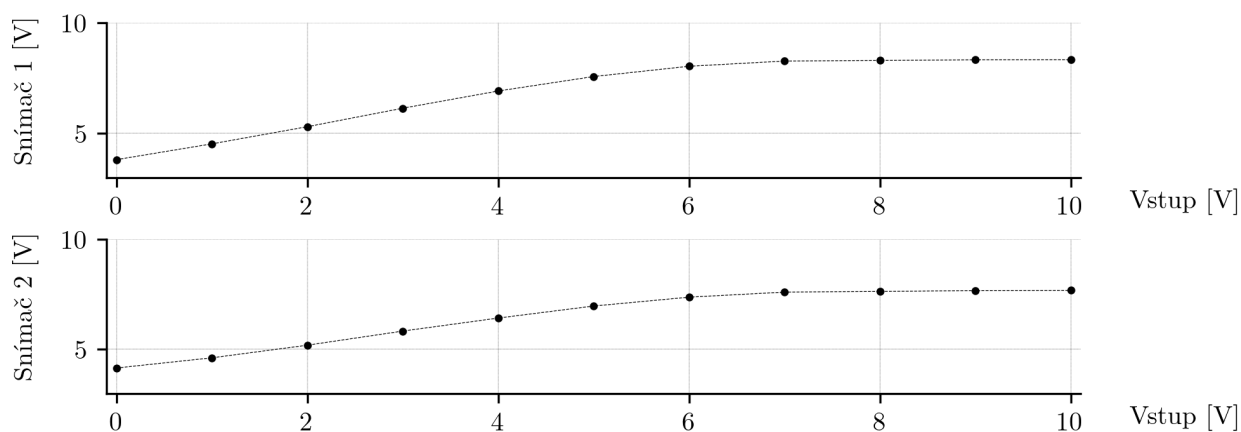
Z tých istých meraní boli zostavené aj prevodové (statické) charakteristiky, ktoré opisujú závislosť výstupných veličín od vstupného napätia špirály pre jednotlivé pracovné body ventilátora.



Obr. 7: Prevodová charakteristika systému TS05 pre pracovný bod ventilátora 3 V



Obr. 8: Prevodová charakteristika systému TS05 pre pracovný bod ventilátora 6 V



Obr. 9: Prevodová charakteristika systému TS05 pre pracovný bod ventilátora 9 V

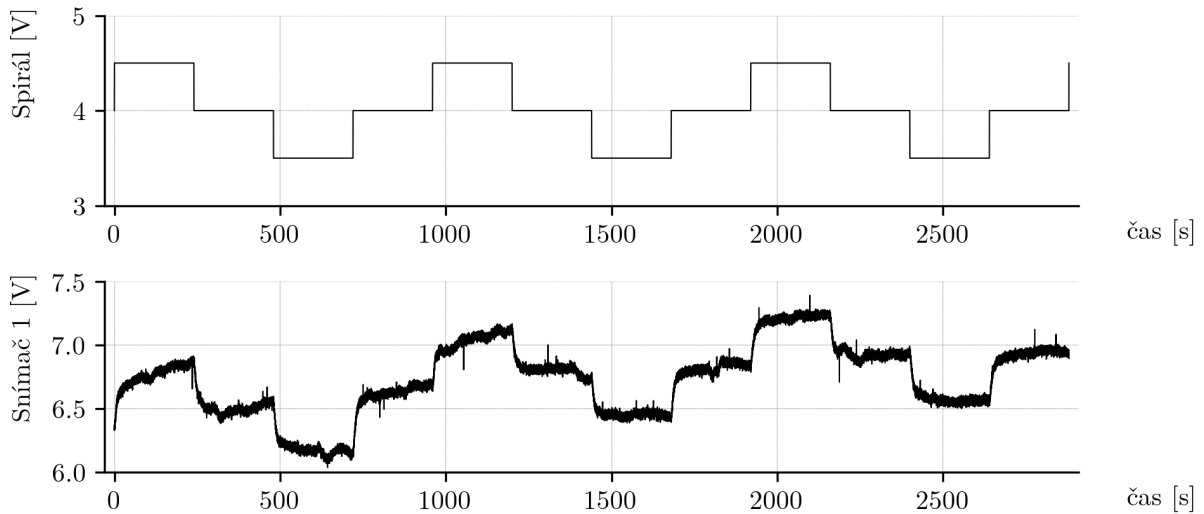
1.2 Identifikácia tepelného systému

V tejto časti sa pokúsime identifikovať tepelný systém TS05 rôznymi metódami a porovnať ich výsledky. Cieľom je určiť, ktoré prístupy poskytujú presnejšie modely a na aké faktory je potrebné dávať pozor počas procesu identifikácie.

1.2.1 Meranie údajov a ich spracovanie

Na účely identifikácie systému akoukoľvek *off-line* metódou je najprv potrebné získať reakciu systému na jednotkové skoky v okolí pracovného bodu. V našom prípade bol pracovný bod zvolený ako vstupné napätie špirály 4 V a ventilátora 6 V.

Na obrázku 10 je zobrazený vstupný signál špirály a odpoveď systému na výstupe zo senzora 1.



Obr. 10: Vstupný signál špirály a výstup senzora 1 pri skokovej zmene

Z grafu možno pozorovať zreteľný lineárny trend, teda postupné zvyšovanie výstupnej teploty počas celého merania. Tento jav je spôsobený pomalým nahrievaním špirály a tiež prirodzenými zmenami teploty v miestnosti, ktoré neboli v rámci experimentu kontrolované. Teplota prostredia sa však počas meraní pohybovala v rozsahu bežnej izbovej teploty, takže systém bol v stabilných podmienkach.

Aby bolo možné získať z nameraných údajov vhodné parametre pre identifikáciu, bolo potrebné odstrániť tento trend aplikovaním lineárnej detrendácie. Metodika výpočtu je založená na lineárnej regresii, kde sa určuje priamka trendu v tvare

$$\hat{y} = \hat{\beta}_0 + \hat{\beta}_1 x \quad (1)$$

pričom koeficienty sa počítajú podľa vzťahov [3]:

$$\hat{\beta}_1 = \frac{n \sum x_i y_i - \sum x_i \sum y_i}{n \sum x_i^2 - (\sum x_i)^2}, \quad \hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x} \quad (2)$$

Takto určený trend je následne odčítaný od pôvodného signálu:

$$y_{\text{detr}} = y - (\hat{\beta}_0 + \hat{\beta}_1 x) \quad (3)$$

Uvedený postup bol implementovaný v prostredí *MATLAB* pomocou vlastnej funkcie:

```
1 function y_detr = my_detrend(y, x)
2     y = y(:);
3     x = x(:);
```

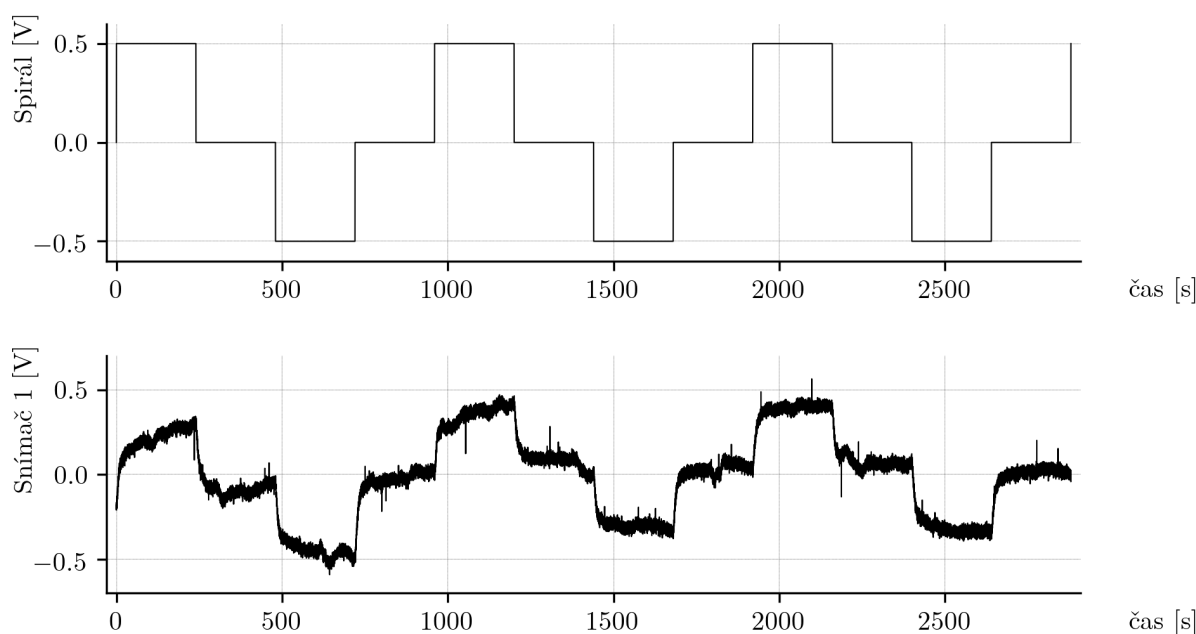
```

4  n = length(y);
5
6  Sx = sum(x);
7  Sy = sum(y);
8  Sxx = sum(x.^2);
9  Sxy = sum(x .* y);
10
11  b1 = (n*Sxy - Sx*Sy) / (n*Sxx - Sx^2);
12  b0 = mean(y) - b1 * mean(x);
13
14  y_detr = y - (b0 + b1*x);
15  end

```

Výpis kódu 1: MATLAB implementácia detrendácie signálu pomocou lineárnej regresie

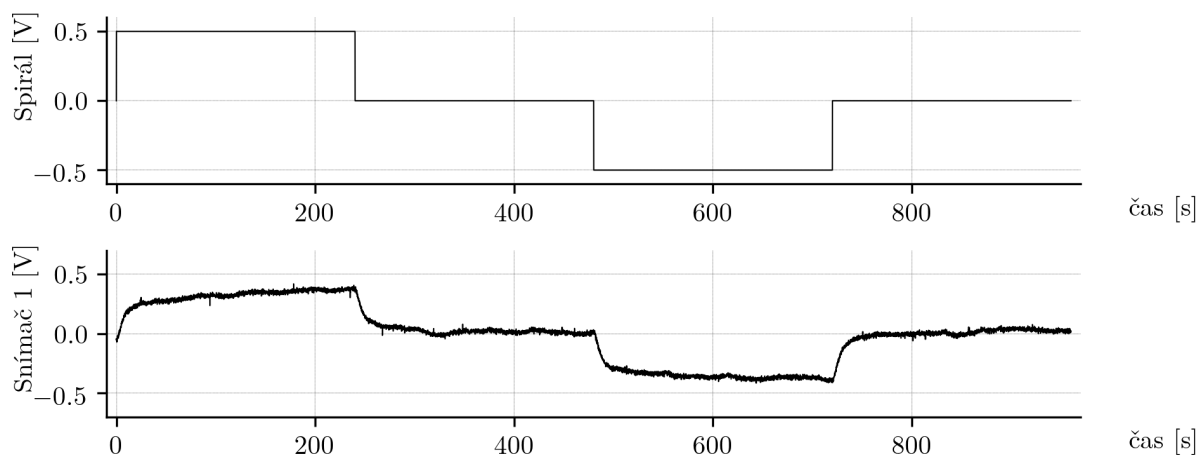
Po aplikovaní detrendácie bol signál zároveň normalizovaný pre potreby ďalšej identifikácie. Výsledné priebehy normalizovaných a detrendovaných údajov sú zobrazené na obrázku 11.



Obr. 11: Normalizované a detrendované priebehy vstupného a výstupného signálu

Z grafov možno vidieť, že jednotlivé skokové odozvy sa mierne líšia svojím priebehom, čo poukazuje na citlivosť systému TS05 na vonkajšie podmienky a fluktuácie prostredia. Preto boli vykonané tri nezávislé experimenty s rovnakým vstupným signálom, ktorých priebehy sú zobrazené na obrázku 11. Po detrendácii a normalizácii boli tieto dáta spriemerované bod po bode, aby sa eliminoval vplyv náhodných rušení a získala reprezentatívna odozva systému.

Výsledný stredný priebeh, znázornený na obrázku 12, predstavuje typickú reakciu systému TS05 v okolí pracovného bodu 4 V s rozsahom $\pm 0,5$ V a slúži ako podklad pre ďalšiu identifikáciu modelu.



Obr. 12: Priemerná odozva systému TS05 v pracovnom bode 4 V

1.2.2 Identifikácia systému pomocou metódy najmenších štvorcov

V tejto časti bola uskutočnená identifikácia systému TS05 na základe priemernej odozvy získanej z troch experimentálnych meraní popísaných v predchádzajúcej kapitole. Tento priemerný priebeh reprezentuje typickú reakciu systému v okolí pracovného bodu 4 V a slúži ako základ pre odhad parametrov modelu. Cieľom bolo získať lineárny model druhého rádu, ktorý dostatočne presne opisuje dynamické správanie systému v tejto oblasti.

Identifikácia bola realizovaná pomocou metódy najmenších štvorcov [1], ktorá umožňuje odhad parametrov diskrétného modelu systému na základe meraných vstupov a výstupov. Pre odhad parametrov bola vytvorená regresná matica $H(k, :)$ a vektor výstupov Y , pričom výsledný odhad parametrov $\hat{\theta}$ bol určený podľa rovníc:

$$H(k, :) = [-y(k-1), -y(k-2), u(k-1), u(k-2)] \quad (4)$$

$$\hat{\theta} = (H^T H)^{-1} H^T Y \quad (5)$$

Na základe týchto odhadov boli vypočítané koeficienty diskrétného modelu druhého rádu, ktorý následne predstavuje prenosovú funkciu $G_d(z)$. Pre účely simulácie a ďalšieho návrhu riadenia bol tento model prevedený na spojitú formu $G(s)$ pomocou Tustinovej aproximácie.

V prostredí MATLAB bola implementovaná funkcia, ktorá realizuje uvedené výpočty a vytvára identifikovaný model systému. Implementácia tejto funkcie je uvedená nižšie:

```

1 function G = ls_identify_basic(y, u, Ts)
2
3     y = y(:);
4     u = u(:);
5     N = length(y);
6     na = 2; nb = 2;
7
8     H = zeros(N-2, na+nb);
9     for k = 3:N
10         H(k-2,:) = [-y(k-1), -y(k-2), u(k-1), u(k-2)];
11     end
12     Y = y(3:N);
13
14     theta = pinv(H) * Y;
15
16     a1 = theta(1);
17     a2 = theta(2);

```

```

18     b1 = theta(3);
19     b2 = theta(4);
20
21     num = [b1 b2];
22     den = [1 a1 a2];
23     Gd = tf(num, den, Ts);
24
25     G = d2c(Gd, 'tustin');
26
27 end

```

Výpis kódu 2: MATLAB implementácia identifikácie metódou najmenších štvorcov

1.2.3 Identifikácia systému pomocou optimalizácie prenosovej funkcie

V tejto časti bol zvolený menej akademický, skôr prakticky orientovaný prístup k identifikácii systému TS05. Namiesto priameho odhadu parametrov modelu metódou najmenších štvorcov bola použitá optimalizácia parametrov prenosovej funkcie tak, aby sa minimalizovala chyba medzi simulovanou odozvou modelu a reálnou priemernou odozvou systému, získanou z experimentov.

Ako nástroj optimalizácie bola použitá funkcia `fminsearch` z prostredia MATLAB, ktorá implementuje Nelderovu–Meadovu simplexovú metódu. Ide o bezderivačnú lokálnu optimalizačnú metódu, ktorá na základe hodnotenia účelovej funkcie v niekoľkých bodoch parametrov iteratívne posúva simplex (mnohouholník v priestore parametrov) tak, aby sa znižovala hodnota účelovej funkcie. V našom prípade je účelová funkcia definovaná ako koreň zo strednej kvadratickej chyby (RMSE) medzi nameranou odozvou systému a simulovaným výstupom modelu:

$$J(\mathbf{p}) = \sqrt{\frac{1}{N} \sum_{k=1}^N (y(k) - y_{\text{sim}}(k, \mathbf{p}))^2} \quad (6)$$

kde $y(k)$ je nameraný výstup, $y_{\text{sim}}(k, \mathbf{p})$ je výstup simulovaný modelom s parametrami $\mathbf{p}$ a N je počet vzoriek merania.

Optimalizácia prebieha nad parametrami diskrétného modelu druhého rádu, ktoré sú následne prevedené na spojitú prenosovú funkciu pomocou Tustinovej aproximácie. Implementácia použitej funkcie v prostredí MATLAB je uvedená nižšie:

```

1 function G = tf_identify_fmin(y, u, time, Ts)
2
3     y = y(:);
4     u = u(:);
5     time = time(:);
6
7     p0 = [0 0 0 0];
8
9     cost_fun = @(p) rmse_cost_stable(p, u, y, time, Ts);
10
11     opts = optimset('Display', 'off', 'TolX', 1e-8, 'TolFun', 1e-8);
12     p_opt = fminsearch(cost_fun, p0, opts);
13
14     p_opt = 50 * tanh(p_opt);
15     a1 = p_opt(1); a2 = p_opt(2); b1 = p_opt(3); b2 = p_opt(4);
16
17     Gd = tf([b1 b2], [1 a1 a2], Ts);
18     G = d2c(Gd, 'tustin');
19
20 end
21
22 function err = rmse_cost_stable(p, u, y, t, Ts)
23
24     p = 50 * tanh(p);
25     a1 = p(1); a2 = p(2); b1 = p(3); b2 = p(4);
26
27     try
28
29         Gd = tf([b1 b2], [1 a1 a2], Ts);

```

```

30
31     poles = pole(Gd);
32     if any(abs(poles) >= 1)
33         err = 1e6;
34         return;
35     end
36
37     G = d2c(Gd, 'tustin');
38     y_sim = lsim(G, u, t);
39
40     err = sqrt(mean((y - y_sim).^2));
41
42     catch
43         err = 1e6;
44     end
45 end

```

Výpis kódu 3: MATLAB implementácia identifikácie pomocou optimalizácie prenosovej funkcie

Pri tomto prístupe je dôležité upozorniť na jeden podstatný problém. Ak nie sú parametre prenosovej funkcie počas optimalizácie vhodne obmedzené, algoritmus môže nájsť také hodnoty parametrov, ktoré síce veľmi dobre aproximujú konkrétnu meranú odozvu, ale vedú k nerealistickému alebo nestabilnému správaniu modelu pri iných vstupoch. V predchádzajúcich pokusoch viedlo práve toto k prenosovým funkciám s „nezdravými“ parametrami, ktoré boli použiteľné iba pre konkrétny fitovaný experiment.

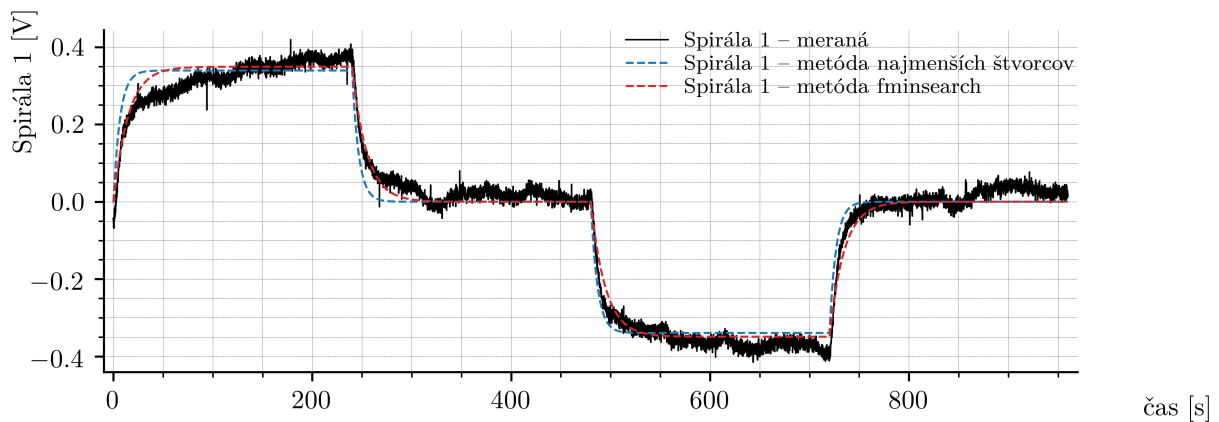
Z tohto dôvodu sú v uvedenej implementácii parametre počas optimalizácie mapované pomocou transformácie

$$p_i^* = 50 \tanh(p_i) \quad (7)$$

čím sú efektívne udržiavané v konečnom rozsahu a znižuje sa riziko vzniku numericky extrémnych alebo fyzikálne nezmyselných modelov. Okrem toho bola do optimalizácie pridaná aj dodatočná penalizácia, ktorá výrazne zvyšuje hodnotu účelovej funkcie v prípade, že identifikovaná prenosová funkcia vykazuje známky nestability. Týmto spôsobom algoritmus uprednostňuje iba stabilné riešenia a zabraňuje vzniku modelov so „zdravotne“ nevhodnými pólmi.

1.2.4 Výsledky identifikácie tepelného systému

Po identifikácii systému v pracovnom bode pomocou dvoch rôznych metód (metódy najmenších štvorcov a optimalizácie prenosovej funkcie) boli získané nasledujúce výsledky.



Obr. 13: Porovnanie skokovej odozvy reálneho systému TS05 a identifikovaných modelov

Porovnanie priemernej skokovej odozvy reálneho systému a odoziev oboch identifikovaných modelov je zobrazené na obrázku 13.

Identifikovaný model získaný metódou najmenších štvorcov má tvar

$$G_{s,LS}(s) = \frac{0.00171 s^2 - 0.3379 s + 6.074}{s^2 + 59.01 s + 8.95} \quad (8)$$

zatým čo model identifikovaný pomocou optimalizácie parametrov prenosovej funkcie metódou *fminsearch* je

$$G_{s,fmin}(s) = \frac{0.0007181 s^2 - 0.05942 s + 0.9012}{s^2 + 18.94 s + 1.291} \quad (9)$$

Kvalita zhody medzi modelmi a experimentálnymi dátami bola hodnotená pomocou ukazovateľa RMSE:

$$RMSE_{LS} = 0.036792, \quad (10)$$

$$RMSE_{fmin} = 0.026996 \quad (11)$$

Ako je vidieť, oba modely poskytujú veľmi uspokojivú aproximáciu dynamiky systému v okolí zvoleného pracovného bodu, pričom model získaný metódou *fminsearch* dosahuje o niečo menšiu hodnotu RMSE. Treba však zdôrazniť, že oba identifikované modely sú len druhého rádu, čo predstavuje rozumný kompromis medzi jednoduchosťou a presnosťou pre následný návrh regulácie.

1.2.5 Aproximácia identifikovaných modelov

Na základe získaných prenosových funkcií možno pozorovať, že ich čitatele obsahujú veľmi malé koeficienty pri prvom a druhom rade. Prítomnosť týchto členov je s veľkou pravdepodobnosťou len výsledkom numerických „artefaktov“ vzniknutých pri prevode z diskkrétnej prenosovej funkcie na spojitú formu. Zahrnutie týchto vyšších rádov v čitateli by výrazne komplikovalo niektoré analytické metódy návrhu regulátorov, napríklad metódu umiestnenia pólov.

Pre účely ďalšej práce sme sa preto rozhodli tieto prenosové funkcie aproximovať jednoduchšou formou. Najzrejmější prístup spočíva v tom, že malé koeficienty pri s a s^2 v čitateli zanedbáme a overíme vplyv tejto úpravy pomocou simulácie.

Výsledkom tejto aproximácie sú nasledujúce prenosové funkcie:

$$G_{s,LS,approx}(s) = \frac{6.074}{s^2 + 59.01 s + 8.95} \quad (12)$$

póly:

$$s_1 = -58.8550, \quad s_2 = -0.1521 \quad (13)$$

a

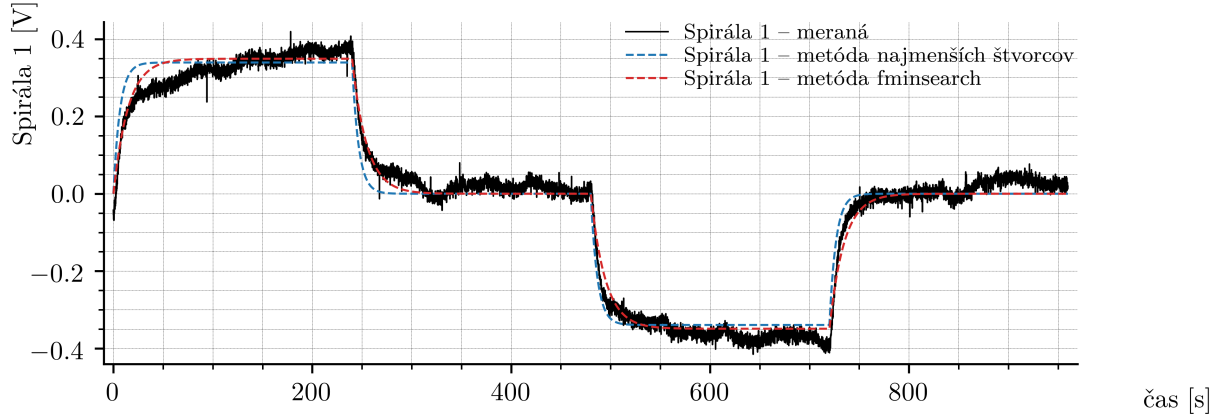
$$G_{s,fmin,approx}(s) = \frac{0.9012}{s^2 + 18.94 s + 1.291} \quad (14)$$

póly:

$$s_1 = -18.8736, \quad s_2 = -0.0684 \quad (15)$$

Z uvedených hodnôt pólov je zrejmé, že obidva modely majú póly s negatívnymi reálnymi časťami, teda ide o stabilné systémy.

Následne bola uskutočnená simulácia odozvy systému analogická tej, ktorá bola uvedená na obrázku 13. Porovnanie reálnej odozvy systému TS05 s oboma aproximovanými modelmi je zobrazené na obrázku 14.



Obr. 14: Porovnanie skokovej odozvy reálneho systému TS05 a aproximovaných modelov

Výsledné hodnoty strednej kvadratickej chyby (RMSE) medzi reálnym systémom a simulovanými odozvami boli:

$$\text{RMSE}_{\text{LS,approx}} = 0.036951, \quad (16)$$

$$\text{RMSE}_{\text{fmin,approx}} = 0.026998 \quad (17)$$

Z výsledkov, ako aj z priebehov simulácií, je zrejmé, že rozdiel medzi pôvodnými a aproximovanými modelmi je zanedbateľný. Hodnota RMSE sa zhoršila len minimálne, čo je z pohľadu ďalšieho použitia modelov úplne prijateľné, najmä vzhľadom na zjednodušenie analytického spracovania, ktoré táto aproximácia prináša.

1.3 PI regulátor pre tepelný systém

V tejto časti je realizovaný PI regulátor pre systém TS05. Pre návrh regulácie bola zvolená prenosová funkcia identifikovaná optimalizačnou metódou najmenších štvorcov (12). Regulátor bol naladený, otestovaný v simulačnom prostredí a následne implementovaný vo forme MATLAB funkcie pre použitie s reálnym systémom.

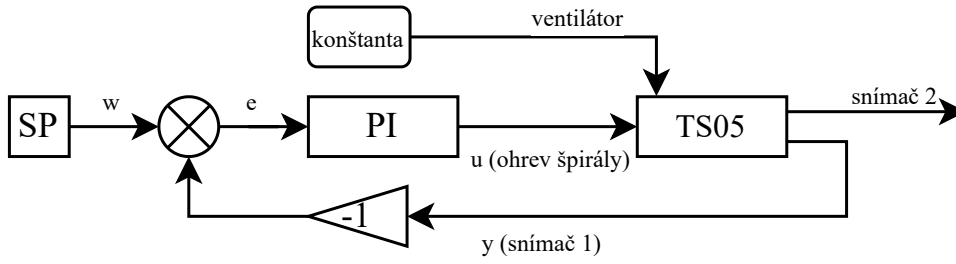
1.3.1 Štruktúra PI regulátora

Pre systém TS05 bol zvolený PI regulátor, pretože nie je potrebné použitie derivačnej časti. D-člen v tepelných systémoch zvyčajne neprináša zlepšenie riadenia, keďže derivácia výrazne zosilňuje merací šum a tepelná dynamika je pomalá, čo robí deriváciu prakticky neefektívnou.

Paralelná forma PI regulátora je

$$C(s) = K_p + \frac{K_i}{s} \quad (18)$$

Regulátor je zapojený v štandardnej uzavretej spätnej väzbe so zápornou spätnou väzbou. Bloková schéma regulácie je uvedená na obrázku 15.



Obr. 15: Bloková schéma uzavretého PI riadenia systému TS05

Tento prístup bol zvolený ako prvý variant implementácie riadenia v reálnom čase, pretože PI regulátor je robustný, ľahko nastaviteľný a jeho výpočetná náročnosť je nízka, čo je výhodné pre implementáciu na mikrokontroléri.

1.3.2 Nastavenie parametrov PI regulátora

Pre získanie parametrov regulátora bola využitá funkcia `pidtune` v prostredí MATLAB, ktorá vykonáva automatizované ladenie regulátora. Tento postup bol zvolený preto, že viaceré analytické metódy ladenia nie je možné na systém TS05 aplikovať.

Charakteristický polynóm uzavretého regulačného obvodu pre PI regulátor má po dosadení tvar

$$s^3 + \frac{b_1}{b_2}s^2 + \left(\frac{b_0}{b_2} + \frac{a_0K_p}{b_2}\right)s + \frac{a_0K_i}{b_2} = 0 \quad (19)$$

Ak by sme chceli použiť metódu umiestnenia pólov pomocou rozšíreného želaného polynómu tretieho rádu, jeho všeobecný tvar je

$$s^3 + (p + 2k\omega_0)s^2 + (2k\omega_0p + \omega_0^2)s + \omega_0^2p = 0 \quad (20)$$

Porovnaním koeficientov polynómov (19) a (20) dostaneme sústavu rovníc:

$$1 = 1 \quad (21)$$

$$p + 2k\omega_0 = \frac{b_1}{b_2} \quad (22)$$

$$2k\omega_0p + \omega_0^2 = \frac{b_0}{b_2} + \frac{a_0K_p}{b_2} \quad (23)$$

$$\omega_0^2p = \frac{a_0K_i}{b_2} \quad (24)$$

Táto sústava nemá realizovateľné riešenie pre parametre K_p a K_i . Dôvodom je, že členy závislé od K_p a K_i sa vyskytujú iba v dvoch rovniciach, zatiaľ čo parametre želaného dynamického správania k , ω_0 , p ovplyvňujú tri rovnice. Sústava je preto pre fyzikálne konzistentné riešenie preurčená a neexistuje v nej žiadna kombinácia parametrov, ktorá by spĺňala všetky rovnice súčasne.

Rovnako nie je možné použiť metódu Ziegler–Nichols, keďže námi identifikovaný model (12) nemá kritický bod, ktorý je pre túto metódu potrebný.

Keďže návrh PI regulátora nie je hlavnou témou práce, bol zvolený numerický nástroj `pidtune`. Jeho princíp spočíva v tom, že pre zvolenú požadovanú šírku pásma uzavretého systému vypočíta optimálne hodnoty parametrov PI regulátora tak, aby bola zabezpečená dobrá dynamická odozva a primeraná robustnosť.

Najskôr bol systém prevedený do diskkrétnej podoby:

```
1 Gs = Gs_fmin;
2 Ts = 0.1;
3
```



```

4 Gz = c2d(Gs, Ts, 'tustin');
5 [Ad,Bd,Cd,Dd] = ssdata(Gz);

```

Následne bolo vykonané ladenie PI regulátora:

```

1 wc = 0.5;
2 Cz = pidtune(Gz, 'pi', wc);
3
4 [numC,denC] = tfdata(Cz,'v');
5 [Ac,Bc,Cc,Dc] = tf2ss(numC,denC);

```

Získané parametre regulátora:

$$K_p = 3.58, \quad K_i = 1.84$$

Prenosová funkcia otvoreného regulačného obvodu je

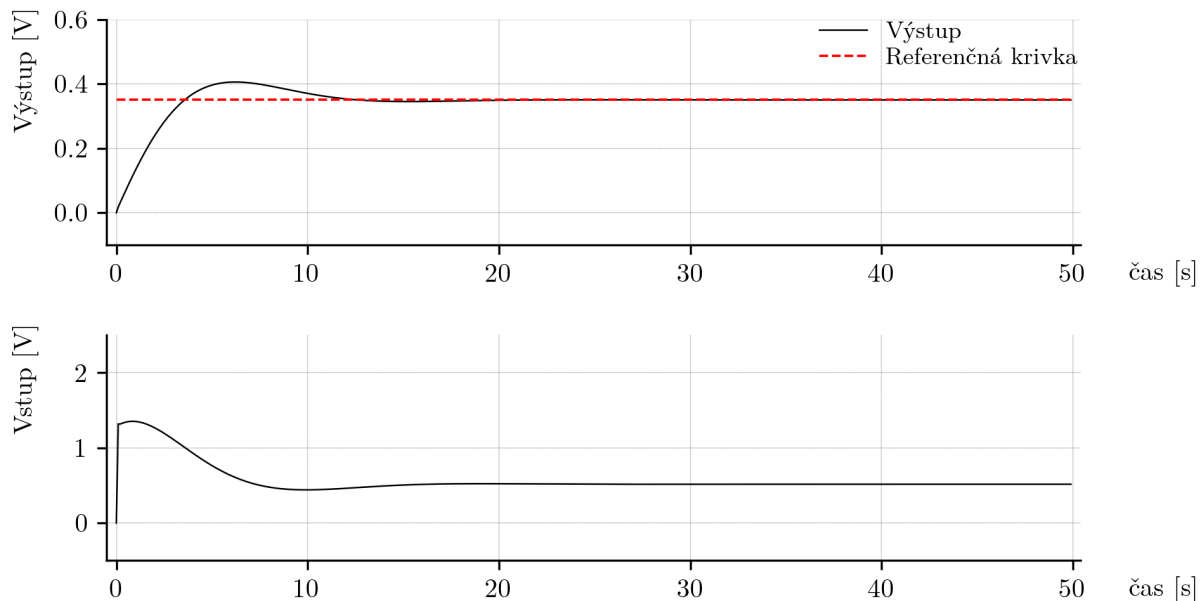
$$G_{oro}(s) = \frac{21.21s + 11.15}{s^3 + 59.01s^2 + 8.95s} \quad (25)$$

Póly otvoreného systému sú:

$$s_1 = 0, \quad s_2 = -58.855, \quad s_3 = -0.1521 \quad (26)$$

Znakom stability je prítomnosť pólov s negatívnou reálnou časťou, čo je splnené.

Simulácia bola vykonaná s obmedzením akčného zásahu na interval $[-4, 6]$, čo zodpovedá pracovnému bodu 4 V. Výsledok simulácie je uvedený na obrázku 16.



Obr. 16: Simulačný výsledok uzavretého PI riadenia systému TS05

Z výsledkov boli namerané tieto hodnoty:

$$\text{Čas ustálenia} = 11.5 \text{ s} \quad (27)$$

$$\text{Prekmit} = 15.9\% \quad (28)$$

$$\text{MSE} = 0.04369 \quad (29)$$

Dosiahnutý výsledok možno považovať za vhodný pre riadenie tepelného systému. Cieľom je dosiahnuť čo najrýchlejšiu odozvu. Prekmit približne 16 percent je akceptovateľný, keďže riadený skok je malý a tepelná sústava je pomalá, čo prirodzene limituje tvar dynamickej odozvy.

1.3.3 Implementácia PI regulátora v MATLABe

Ako je uvedené v zadaní, cieľom práce je implementácia riadiacich algoritmov na rôznych hardvérových a softvérových platformách. Preto bol PI regulátor implementovaný tak, aby mohol pracovať v reálnom čase bez použitia hotových Simulink blokov určených na riadenie. Celá logika riadenia bola zapísaná priamo do bloku MATLAB Function, ktorý sa vykonáva raz za každý riadiaci cyklus s periódou vzorkovania 0.1 sekundy.

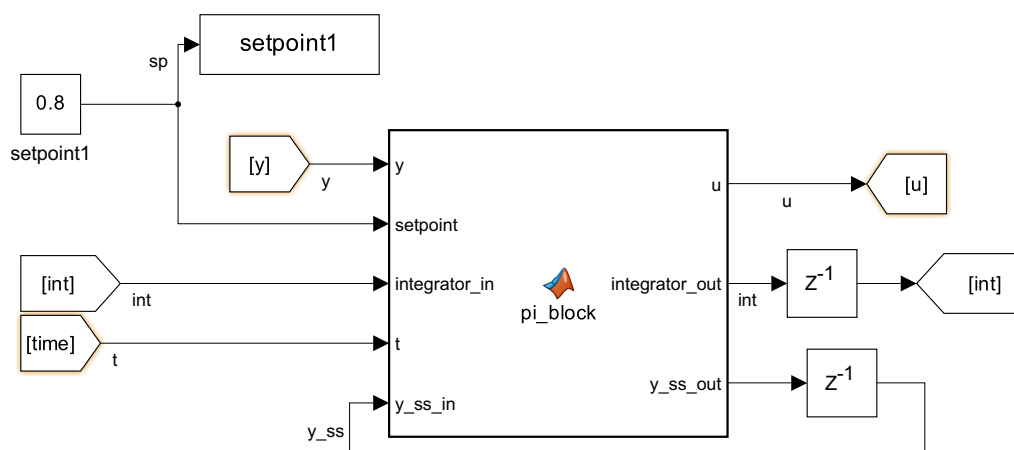
Implementácia regulátora je nasledovná:

```
1 function [u, integrator_out, y_ss_out] = ...
2     pi_block(y, setpoint, integrator_in, t, y_ss_in)
3
4 % PI controller
5
6 pb_u = 4;
7 Ts = 0.1;
8 u_min = - pb_u;
9 u_max = 10 - pb_u;
10
11 Kp = 3.58;
12 Ki = 1.84;
13
14 % waiting phase for steady-state detection
15 if t < 10*60
16     u = 4;
17     y_ss_out = y;
18     integrator_out = 0;
19
20 else
21     y_ss_out = y_ss_in;
22     pb_y = y_ss_in;
23
24     % output normalization
25     y = y - pb_y;
26
27     % control error
28     e = setpoint - y;
29
30     % integrator
31     integrator_out = integrator_in + e * Ts;
32
33     % PI control law
34     u = Kp * e + Ki * integrator_out;
35     u = min(max(u, u_min), u_max);
36
37     % input normalization
38     u = u + pb_u;
39
40 end
41 end
```

Tento blok však na rozdiel od štandardných Simulinkových riadiacich blokov neobsahuje vlastnú vnútornú pamäť, preto je potrebné stavové premenné prenášať medzi jednotlivými volaniami bloku explicitne.

Z tohto dôvodu sú hodnoty integrátora a odhad ustáleného stavu odosielané na výstup PI bloku a v nasledujúcom kroku sú privádzané späť na vstup. Týmto spôsobom sa zabezpečí, že regulátor si zachováva svoje vnútorné stavy z jedného riadiaceho kroku do druhého.

Logika zapojenia bloku MATLAB Function je zobrazená na obrázku 17, ktorý ukazuje vstupy, výstupy a spätnú väzbu stavových premenných potrebných pre správnu činnosť regulátora.



Obr. 17: Zapojenie PI regulátora v bloku MATLAB Function so spätnou väzbou stavov

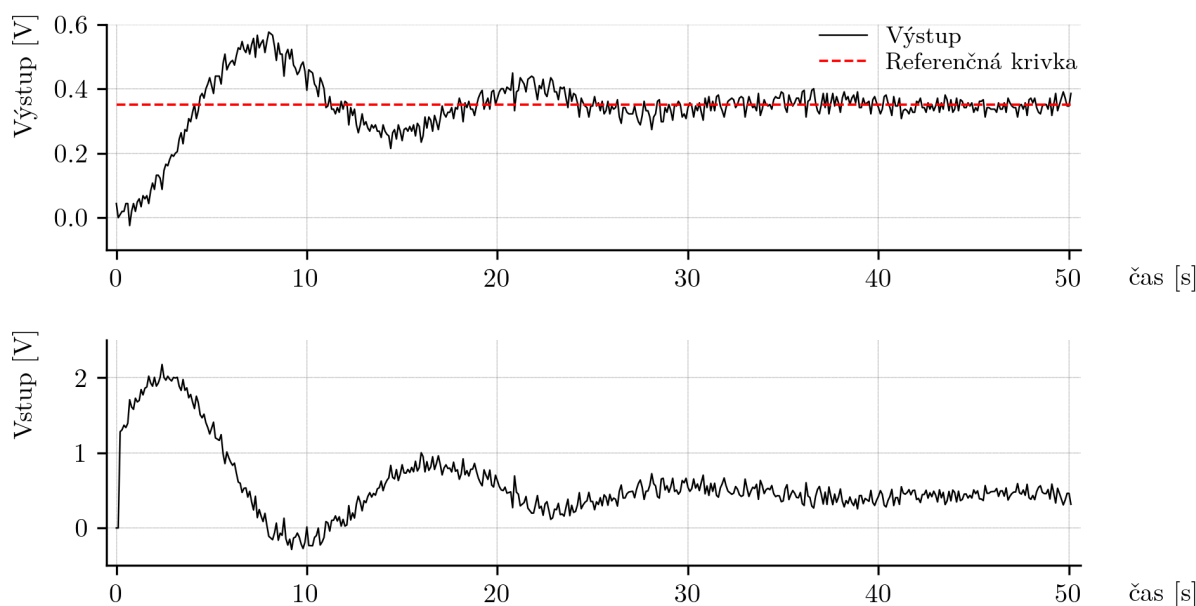
Pred samotným začiatkom regulácie bol zavedený časový úsek, počas ktorého regulátor nevykonáva žiadny zásah. Počas prvých desiatich minút sa na výstupe drží hodnota pracovného bodu a zároveň sa sleduje teplota na výstupe systému. Cieľom je získať reálnu hodnotu ustáleného stavu priamo z aktuálnych podmienok experimentu.

Použitie hodnôt zosadených zo statických charakteristík (obr. 8) nie je vhodné. Tepelný systém je citlivý na teplotu v miestnosti a experimenty nebolo možné realizovať v rovnakých podmienkach. Okrem toho surové merania (obr. 10) ukázali, že teplota narastá veľmi pomaly aj počas dlhého experimentu, čo znamená, že systém nemá pevnú, opakovateľnú hodnotu ustáleného stavu.

Z toho dôvodu je ustálený stav zisťovaný automaticky priamo regulátorom ako posledná nameraná hodnota y pred spustením PI riadenia. Tento prístup funguje bez ohľadu na to, či je systém iba málo zahriaty alebo už dosiahol vysokú prevádzkovú teplotu.

Experiment 1: Regulácia okolo jedného pracovného bodu

Na základe implementácie uvedenej vyššie bol následne vykonaný experiment v simulačnom prostredí s cieľom overiť kvalitu regulácie. Priebeh regulácie je zobrazený na obrázku 18.



Obr. 18: Simulačný experiment s PI regulátorom pri skokovej zmene žiadanej hodnoty

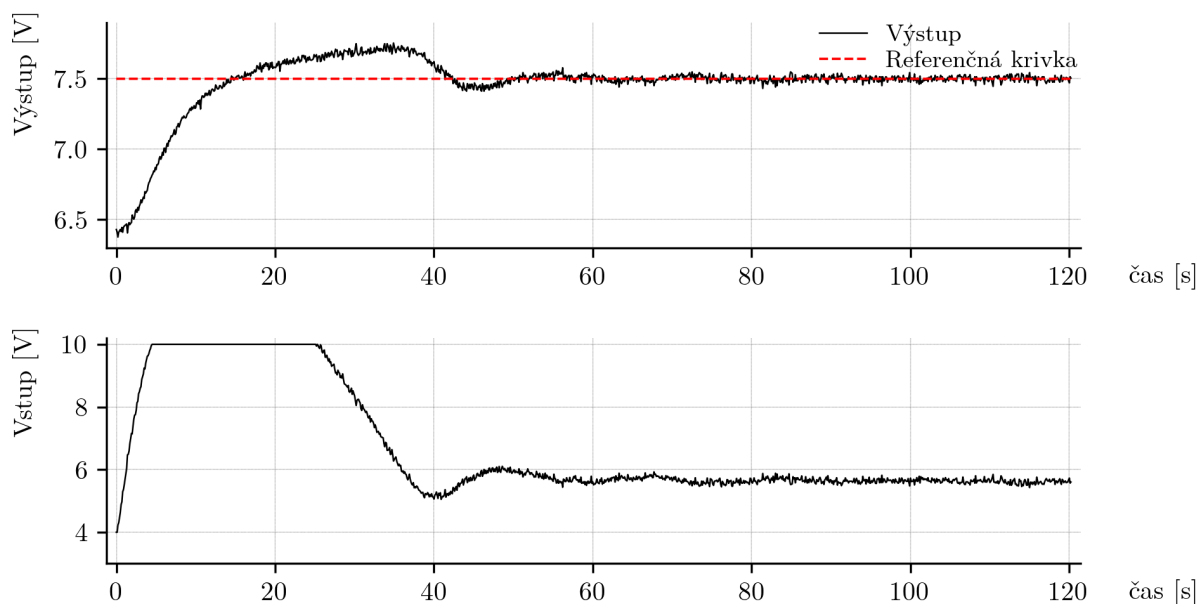
Kvalita regulácie bola hodnotená podľa troch kritérií:

- prekmit: žiadaná hodnota bola 0.35 V, maximálna hodnota výstupu dosiahla približne 0.52 V, čo zodpovedá približne 49.0 percentnému prekmitu,
- čas ustálenia: výstup sa ustálil približne po 28 sekundách,
- trvalá regulačná odchýlka: prakticky nulová.

Pri porovnaní so simuláciou uvedenou v predchádzajúcich kapitolách možno vidieť, že prekmit je výrazne vyšší. Dôvodom je skutočnosť, že identifikované modely nedokázali zachytiť celú dynamiku systému TS05 a viacero pomalých javov, ktoré sa prejavujú až pri reálnej prevádzke. Napriek tomu je regulačný čas veľmi priaznivý vzhľadom na to, že ide o pomalý tepelný systém. Rýchla regulácia bola prioritou, preto možno výsledok považovať za uspokojivý pre riadenie okolo jedného pracovného bodu.

Experiment 2: Reakcia na väčší skok

Nasledoval druhý experiment, ktorého cieľom bolo overiť správanie regulátora pri väčšej skokovej zmene žiadanej hodnoty, ktorá má simulovať prechod medzi rôznymi pracovnými bodmi systému. Systém bol najprv ustálený v pracovnom bode s výstupnou hodnotou približne 6.4 V a následne bola žiadaná hodnota skokovo zmenená na 7.5 V. Výsledok je zobrazený na obrázku 19.



Obr. 19: Experiment PI regulátora pri skoku z 6.4 V na 7.5 V

Z grafu je zrejmé, že počas regulácie došlo k výraznému presýteniu akčného zásahu. Ovládací signál dosiahol svoj limit a počas určitého času na ňom zotrval. Aj keď bola požadovaná hodnota dosiahnutá, takéto správanie je z pohľadu bezpečnosti a kvality regulácie nevyhovujúce.

Z tohto dôvodu možno konštatovať, že navrhnutý PI regulátor je vhodný na riadenie v blízkosti jedného pracovného bodu, avšak nie je vhodný na prepínanie medzi vzdialenejšími pracovnými bodmi. V ďalšej časti práce bude preto implementovaný postup umožňujúci bezpečné a stabilné prepínanie medzi pracovnými bodmi a následne budú jednotlivé regulátory skombinované tak, aby systém disponoval širším využiteľným rozsahom riadenia.

1.4 Prepínač pracovných bodov

V tejto časti sa budeme zaoberať návrhom a testovaním prepínača pracovných bodov. Prepínač bude predstavovať pomalý regulátor, ktorého úlohou bude zabezpečiť robustné prechody medzi rôznymi pracovnými bodmi systému TS05. Cieľom je vytvoriť taký spôsob riadenia, ktorý umožní spoľahlivý presun z ľubovoľného pracovného bodu do ľubovoľného iného pracovného bodu bez neželaných dynamických javov, ako je presýtenie akčného zásahu alebo príliš veľké prechodové správanie.

Okrem samotného prepínača bola v tejto časti rozšírená aj pracovná oblasť regulácie o dva ďalšie pracovné body systému – pre napätia špirály 2 V a 6 V. Pre každý z týchto pracovných bodov bol postup opakovaný presne podľa postupu uvedeného v sekcii ??, teda boli:

- vykonané merania reakcií systému na jednotkové skoky ± 0.5 V okolo pracovného bodu,
- z nameraných údajov bola identifikovaná prenosová funkcia v danom pracovnom bode,
- pre každý pracovný bod bol navrhnutý PI regulátor s vlastnými parametrami,
- boli určené hodnoty y_{pb} a rozsah výstupu pre ± 0.5 V zmenu vstupu.

Prehľad všetkých troch pracovných bodov je uvedený v tabuľke 2, kde sú uvedené identifikované prenosové funkcie, hodnoty pracovného bodu y_{pb} a parametre jednotlivých PI regulátorov.

Tabuľka 2: Parametre pracovných bodov, identifikované prenosové funkcie a PI regulátory

u_{pb}	Prenosová funkcia	y_{pb}	Δy_{pb} pri $u_{pb} \pm 0.5$ V	PI parametre
2 V	$\frac{5.79}{s^2 + 44.21s + 7.276}$	4.4884	± 0.41	$K_p = 2.77, K_i = 1.48$
4 V	$\frac{6.074}{s^2 + 59.01s + 8.95}$	6.3412	± 0.35	$K_p = 3.58, K_i = 1.84$
6 V	$\frac{7.161}{s^2 + 52.87s + 21.37}$	7.7441	± 0.18	$K_p = 1.83, K_i = 2.20$

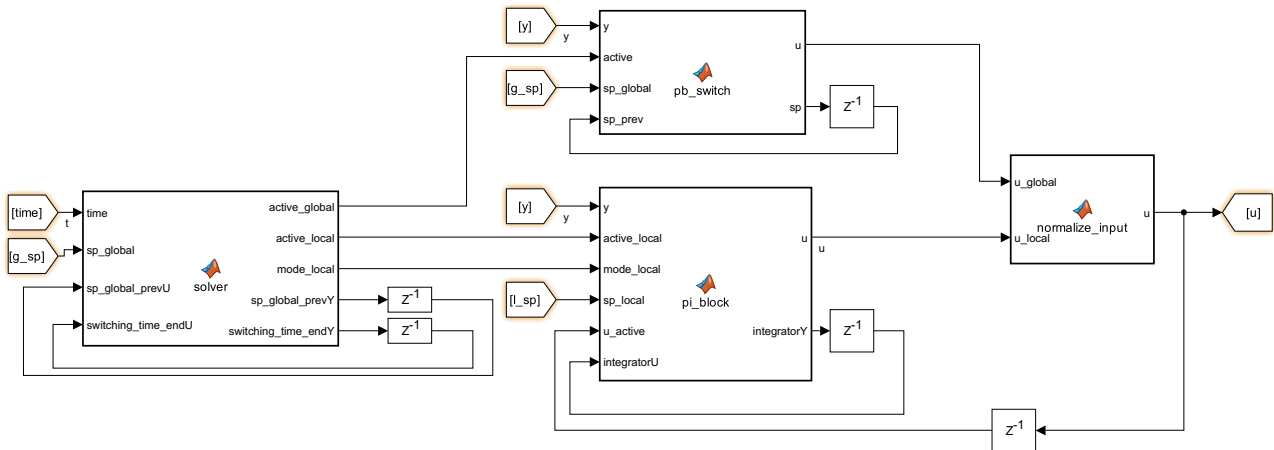
Keďže pôvodné statické charakteristiky systému boli merané ešte počas letného obdobia, bolo rozhodnuté hodnoty y_{pb} pre tieto experimenty zmerať znova, rovnakým spôsobom ako v sekcii ?? . V tabuľke sú preto uvedené aktuálne hodnoty pracovných bodov, ktoré vernejšie reprezentujú stav systému počas meraní v zimnom období.

1.4.1 Opis a schéma riadenia z použitím prepínača

V tejto časti sa prezentuje návrh riadenia systému TS05 s využitím prepínača pracovných bodov. Zámerom riadenia je pokryť dve odlišné situácie:

- reguláciu v okolí zvoleného pracovného bodu.
- bezpečný prechod medzi pracovnými bodmi bez neželaných javov, ako je dlhodobé presýtenie akčného zásahu alebo príliš agresívna dynamika.

Navrhnutá štruktúra je založená na tom, že pri zmene globálnej žiadanej hodnoty má prioritu prechodový režim (prepínač pracovných bodov), a až po jeho ukončení sa aktivuje lokálna PI regulácia v okolí aktuálneho pracovného bodu. Schéma celého riadenia je uvedená na obr. 20.



Obr. 20: Schéma riadenia systému TS05 s prepínačom pracovných bodov

Ako je možné vidieť na schéme, rozhodovacia logika je sústredená v bloku *solver*, ktorý na základe globálnej žiadanej hodnoty určuje, či sa systém nachádza v režime prechodu medzi pracovnými bodmi, alebo už môže prebiehať lokálna regulácia okolo pracovného bodu. Okrem toho blok *solver* nepretržite určuje aj aktuálny režim *mode_local*, t. j. informáciu o tom, ku ktorému z pracovných bodov (2 V, 4 V alebo 6 V na špirále) má byť priradená lokálna regulácia.

Implementácia bloku *solver* je uvedená v kóde 4. Základom je výpočet času, počas ktorého má byť aktívny prechodový režim. Tento čas sa odvodzuje z veľkosti zmeny

globálnej žiadanej hodnoty a z kroku zjemnenia *step_to_soft*, pričom sa uvažuje aj dodatočná časová rezerva *extra_time*.

```

1 function [active_global, active_local, mode_local, sp_global_prevY,
   switching_time_endY] = solver(time, sp_global, sp_global_prevU,
   switching_time_endU)
2
3 %Constants
4 step_to_soft = 0.005;
5 Ts = 0.1;
6 extra_time = 10;
7 eps = 1e-5;
8
9 active_global = 0;
10 active_local = 0;
11
12 %Time of switch working
13 if abs(sp_global - sp_global_prevU) > eps
14     reg_time = (abs(sp_global - sp_global_prevU) / step_to_soft) * Ts + extra_time;
15     reg_time = max(reg_time, extra_time);
16     switching_time_endY = time + reg_time;
17 else
18     switching_time_endY = switching_time_endU;
19 end
20
21 %Activators of PI and switch
22 if time < switching_time_endY
23     active_global = 1;
24     active_local = 0;
25 else
26     active_global = 0;
27     active_local = 1;
28 end
29
30 if sp_global > 7.6
31     mode_local = 6;
32 elseif sp_global > 6.2
33     mode_local = 4;
34 else
35     mode_local = 2;
36 end
37
38 %memory
39 sp_global_prevY = sp_global;
40
41 end

```

Výpis kódu 4: MATLAB implementácia rozhodovacej logiky prepínača pracovných bodov (*solver*)

Z kódu 4 vyplýva, že dĺžka prechodového režimu je určená vzťahom

$$t_{\text{reg}} = \left(\frac{|sp_{\text{global}} - sp_{\text{global,prev}}|}{\Delta sp} \right) T_s + t_{\text{extra}}, \quad t_{\text{end}} = t + t_{\text{reg}}, \quad (30)$$

kde Δsp predstavuje krok zjemnenia (*step_to_soft*), T_s je vzorkovací čas a t_{extra} je časová rezerva. Prechodový režim má v navrhutej štruktúre prioritu, t.j. počas jeho trvania je lokálna PI regulácia zablokovaná.

Po rozhodnutí o režime nasledujú samotné regulátory. V ľubovoľnom časovom okamihu je aktívny vždy iba jeden z nich: buď prechodový regulátor (prepínač pracovných bodov), alebo lokálny PI regulátor. Prepínač pracovných bodov *pb_switch* realizuje zjemnenie žiadanej hodnoty tak, aby namiesto skoku vznikla rampa, čo výrazne znižuje riziko presýtenia akčného zásahu pri veľkých zmenách referencie. Zjemnenie prebieha s krokom $\Delta sp = 0.005$ na jednu iteráciu, t.j. požadovaná hodnota sa mení postupne, kým nedosiahne cieľovú globálnu hodnotu. Zjemnený setpoint je následne použitý v jednoduchom P regulátore.

Kód bloku *pb_switch* je uvedený v kóde 5.

```

1 function [u, sp] = pb_switch(y, active, sp_global, sp_prev)
2

```

```

3      %Constants
4      Kp = 4;
5      Ts = 0.1;
6      step_to_soft = 0.005;
7
8      %Logic
9      if active
10
11         if abs(sp_global - sp_prev) > step_to_soft
12             delta = sign(sp_global - sp_prev) * step_to_soft;
13         else
14             delta = sp_global - sp_prev;
15         end
16
17         sp = sp_prev + delta;
18         u = Kp * (sp - y);
19
20     else
21
22         sp = sp_global;
23         u = 0;
24
25     end
26
27     u = min(max(u,0), 10);
28
29 end

```

Výpis kódu 5: MATLAB implementácia prepínača pracovných bodov (*pb_switch*)

Čas, počas ktorého je *pb_switch* aktívny, je priamo viazaný na veľkosť zmeny globálneho setpointu, čo bolo definované v bloku *solver* vzťahom (30). Po uplynutí tejto doby sa aktivuje lokálny PI regulátor, ktorý zabezpečuje presné sledovanie žiadanej hodnoty v okolí pracovného bodu.

Lokálna PI regulácia je realizovaná blokom *pi_block*, ktorý obsahuje dve dôležité úpravy:

- ziskové plánovanie (gain scheduling) podľa aktuálne zvoleného pracovného bodu,
- sledovanie integrátora aj v neaktívnom stave (tzv. bumpless transfer), aby po opätovnej aktivácii nevznikol neželaný skok akčného zásahu.

Ziskové plánovanie spočíva v tom, že podľa signálu *mode_local* (určeného v *solver*) sa zvolia príslušné parametre K_p , K_i a hodnoty pracovného bodu (u_{pb} , y_{pb}). Normalizácia prebieha odčítaním y_{pb} od meraného výstupu a akčný zásah je limitovaný v rozsahu okolo u_{pb} .

Implementácia *pi_block* je uvedená v kóde 6.

```

1 function [u, integratorY] = pi_block(y, active_local, mode_local, sp_local, u_active,
2     integratorU)
3
4     Ts = 0.1;
5
6     %Gain scheduling
7     if mode_local == 2
8         pb_u = 2; pb_y = 4.4884;
9         Kp = 2.77; Ki = 1.48;
10    elseif mode_local == 4
11        pb_u = 4; pb_y = 6.3412;
12        Kp = 3.58; Ki = 1.84;
13    else
14        pb_u = 6; pb_y = 7.7441;
15        Kp = 1.83; Ki = 2.20;
16    end
17
18    u_min = 0 - pb_u;
19    u_max = 10 - pb_u;
20
21    yN = y - pb_y;
22    e = sp_local - yN;

```



```

22
23     if active_local == 0
24         uN = u_active - pb_u;
25         uN = min(max(uN, u_min), u_max);
26         integratorY = (uN - Kp*e) / Ki;
27         u = 0;
28     else
29         integratorY = integratorU + e*Ts;
30         uN = Kp*e + Ki*integratorY;
31         uN = min(max(uN, u_min), u_max);
32         u = uN + pb_u;
33     end
34
35 end

```

Výpis kódu 6: MATLAB implementácia lokálnej PI regulácie so ziskovým plánovaním a sledovaním integrátora (*pi_block*)

Sledovanie integrátora v neaktívnom stave je realizované tak, že sa z rovnice PI regulátora v normalizovanom tvare

$$u_N = K_p e + K_i I \quad (31)$$

vyjadrí stav integrátora

$$I = \frac{u_N - K_p e}{K_i}. \quad (32)$$

Týmto spôsobom je zabezpečené, že pri prepnutí režimu nedôjde k náhlej zmene vnútornej integrujúcej zložky, a teda ani k nežiadanému skoku na výstupe regulátora.

Výstupy prechodového regulátora a lokálneho PI regulátora sa následne kombinujú v bloku *normalize_input*, kde sa vykoná súčet a finálne obmedzenie signálu do rozsahu 0–10 V. Keďže v každom okamihu je aktívny iba jeden regulátor (druhý dáva nulový výstup), je táto forma kombinácie v danej štruktúre prípustná a nevedie k nejednoznačnosti riadenia.

Kód bloku *normalize_input* je uvedený v kóde 7.

```

1 function u = normalize_input(u_global, u_local)
2
3     u_sum = u_global + u_local;
4     u = min(max(u_sum, 0), 10);
5
6 end

```

Výpis kódu 7: MATLAB implementácia finálnej normalizácie a obmedzenia akčného zásahu (*normalize_input*)

Literatúra

- [1] M. Fikar, J. Mikleš, *Identifikácia systémov*. Slovenská technická univerzita v Bratislave, 1998. ISBN 80-227-1177-2.
- [2] M. Tárník, Učebný text KUT015: *Laboratórne zariadenie TS: orientačný prehľad*. Slovenská technická univerzita v Bratislave, 2025. Dostupné online: <https://github.com/OkoliePracovnehoBodu/KUT/>
- [3] D. C. Montgomery, E. A. Peck, G. G. Vining, *Introduction to Linear Regression Analysis, Fifth Edition*. John Wiley & Sons, Inc., Hoboken, New Jersey, 2012. ISBN 978-0-470-54281-1.